

ReportaTuCalle: Plataforma Web para el Registro y Atención de Incidencias Urbanas

Bryan Aldrin Chontay Campos, Paul Fran León Gonzales, Arturo Miguel Osorio Miranda,
Erick Joel Sánchez Longa y Joaquín Marcelo Torres Reátegui
Universidad Nacional de Ingeniería
Lima, Perú

Resumen—Este trabajo presenta ReportaTuCalle, una plataforma web orientada al registro y seguimiento de incidencias urbanas. El sistema permite que los ciudadanos reporten problemas como baches, semáforos averiados, basura acumulada, fugas de agua o veredas deterioradas mediante una descripción, una imagen y una ubicación geográfica. A su vez, los supervisores pueden atender incidencias asignadas y consultar rutas de trabajo, mientras que el administrador gestiona usuarios, categorías y asignaciones. La solución fue desarrollada con React para el cliente web y Spring Boot para el backend, utilizando PostgreSQL con PostGIS para la información geográfica y Cloudinary para el manejo de imágenes. Además, se empleó una organización basada en arquitectura hexagonal y se aplicaron patrones de diseño como Strategy, State, Observer, Adapter y Builder. También se incorporaron pruebas unitarias, funcionales y de carga. El resultado es un prototipo funcional que centraliza la gestión de reportes y mejora el seguimiento del proceso de atención.

Index Terms—incidencias urbanas, aplicación web, arquitectura hexagonal, Spring Boot, PostgreSQL, patrones de diseño, optimización de rutas

I. INTRODUCCIÓN

En muchas ciudades, los problemas de la vía pública suelen comunicarse mediante llamadas, mensajes o publicaciones dispersas en redes sociales. Aunque estos medios permiten informar que existe una incidencia, no siempre facilitan conocer su ubicación exacta, verificar quién debe atenderla o comprobar si ya fue resuelta. Además, cuando la información se encuentra distribuida entre diferentes canales, el seguimiento se vuelve más difícil tanto para el ciudadano como para el personal encargado.

Con base en esta necesidad se desarrolló **ReportaTuCalle**, una plataforma web que concentra los reportes ciudadanos en un mismo sistema. Cada incidencia puede registrarse con una categoría, una descripción, coordenadas y evidencia fotográfica. Posteriormente, el personal municipal puede revisar los casos pendientes, asignarlos a un supervisor y actualizar su estado conforme avanza la atención.

El objetivo principal fue construir un prototipo funcional que permita organizar el ciclo completo de un reporte. Para documentar este alcance se consideró una separación entre requisitos funcionales y no funcionales, de acuerdo con las recomendaciones generales de ingeniería de requisitos de ISO/IEC/IEEE 29148 [1].

Otro objetivo fue mantener el código organizado. Por este motivo, el backend se estructuró siguiendo principios de arquitectura hexagonal, de forma que la lógica del negocio

no dependa directamente de la base de datos o de servicios externos.

II. PROBLEMA Y ALCANCE DE LA PROPUESTA

II-A. Problema identificado

Una incidencia urbana puede permanecer sin atención cuando la información llega incompleta o no existe un canal claro de seguimiento. Por ejemplo, un ciudadano puede informar sobre un bache, pero no proporcionar una dirección precisa; también puede ocurrir que una fotografía se comparta sin indicar si el problema continúa activo. En otros casos, diferentes trabajadores pueden dirigirse a una misma zona mientras existen reportes pendientes en otros lugares.

La falta de una plataforma centralizada dificulta responder preguntas básicas: qué ocurrió, dónde ocurrió, quién lo reportó, quién debe atenderlo y en qué estado se encuentra. Estas preguntas forman parte del flujo que el sistema busca ordenar.

II-B. Alcance del prototipo

El prototipo desarrollado considera tres tipos de usuario y cubre desde el registro del incidente hasta su resolución. La propuesta incluye autenticación, visualización geográfica, administración de categorías, asignación de reportes y consulta de rutas. También se considera la notificación de cambios de estado y el almacenamiento externo de imágenes.

No se plantea reemplazar todos los procesos internos de una municipalidad. El sistema funciona como una base de trabajo sobre la cual podrían integrarse en el futuro mecanismos de priorización, estadísticas de atención, validación de evidencia o interoperabilidad con otras plataformas.

III. REQUISITOS Y USUARIOS DEL SISTEMA

El sistema define tres roles principales: ciudadano, supervisor y administrador. Cada uno cuenta con permisos distintos para evitar accesos indebidos y para mantener un flujo de trabajo claro dentro de la aplicación.

Cuadro I: Funciones principales por rol

Rol	Funciones principales
Ciudadano	Registrar incidencias, adjuntar evidencia fotográfica, consultar sus reportes y visualizar incidencias en el mapa.
Supervisor	Revisar reportes pendientes o asignados, actualizar el estado de atención y consultar una ruta de trabajo.
Administrador	Gestionar usuarios, supervisores, categorías y asignaciones de reportes.

III-A. Requisitos funcionales principales

Los requisitos funcionales se agruparon en módulos para facilitar su implementación. En autenticación se consideraron el registro, el inicio de sesión y el control de acceso mediante roles. En reportes se incluyeron la creación, consulta, asignación, actualización y cambio de estado. En categorías se implementó la gestión del catálogo utilizado por los ciudadanos al registrar una incidencia.

El módulo de medios permite adjuntar imágenes de evidencia, mientras que el módulo de optimización se encarga de calcular recorridos para un conjunto de incidencias. El módulo de notificaciones permite informar cambios relevantes sin que el usuario tenga que recargar constantemente la página.

Cuadro II: Módulos funcionales del sistema

Módulo	Responsabilidad
Autenticación	Registro, inicio de sesión, validación de token y control de acceso.
Usuarios	Gestión del perfil, rol y datos básicos de la cuenta.
Reportes	Creación, consulta, asignación y actualización de incidencias.
Categorías	Mantenimiento de tipos de incidencia y asociación con reglas del sistema.
Multimedia	Subida y consulta de imágenes de evidencia.
Notificaciones	Aviso de cambios de estado y eventos importantes.
Optimización	Cálculo de rutas o estructuras de conexión según la categoría.

III-B. Flujo general

El flujo principal comienza cuando un ciudadano registra un reporte. El sistema valida la información, almacena la imagen y registra la ubicación. El reporte queda disponible para administración o supervisión. Luego puede ser asignado, marcado como en proceso y finalmente resuelto. Cada cambio debe respetar el estado actual del reporte para evitar transiciones inconsistentes.

IV. ARQUITECTURA Y TECNOLOGÍAS

El backend fue organizado como un monolito modular con principios de arquitectura hexagonal. Esta arquitectura propone ubicar la lógica de la aplicación en el centro y conectarla con interfaces externas mediante puertos y adaptadores [2]. Así se evita que las reglas del negocio dependan directamente de controladores, persistencia o proveedores externos.

La Fig. 1 muestra una vista simplificada de la arquitectura propuesta.

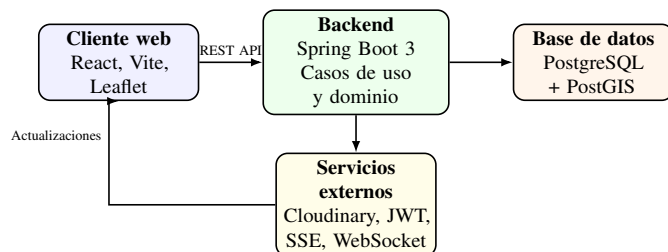


Figura 1: Vista simplificada de la arquitectura del sistema.

IV-A. Separación por capas

En la capa de entrada se encuentran los controladores REST y los mecanismos de comunicación en tiempo real. Los controladores reciben solicitudes, validan datos básicos y delegan la ejecución a los casos de uso. La lógica principal se ubica en la capa de aplicación y dominio, donde se manejan reportes, usuarios, categorías y estrategias de optimización.

La capa de salida contiene adaptadores para persistencia, almacenamiento de imágenes y servicios externos. En lugar de hacer que el dominio conozca directamente JPA, Cloudinary o un cliente HTTP, se utilizan puertos e interfaces. De esta manera, una implementación puede sustituirse sin modificar toda la aplicación.

IV-B. Tecnologías utilizadas

Cuadro III: Tecnologías utilizadas

Área	Tecnologías
Cliente web	React 18, Vite, Tailwind CSS, Leaflet y React Router.
Backend	Java 21, Spring Boot 3, Spring Security, JWT y JPA/Hibernate.
Datos	PostgreSQL y PostGIS para coordenadas e información geográfica.
Multimedia	Cloudinary para almacenar imágenes de evidencia.
Tiempo real	Server-Sent Events y WebSocket.
Pruebas	JUnit, Mockito, Cypress y Locust.
Despliegue	Vercel, Render y Supabase.

IV-C. Base de datos y datos geográficos

La base de datos utiliza PostgreSQL junto con PostGIS para almacenar y consultar información espacial [3]. Cada reporte contiene una ubicación representada mediante un punto. Este enfoque permite calcular distancias, buscar incidencias cercanas y mostrar los resultados en un mapa.

La información principal de un reporte incluye su descripción, categoría, estado, usuario que lo registró, supervisor asignado, imagen y coordenadas. La separación por módulos ayuda a mantener organizadas las tablas de autenticación, usuarios, reportes, categorías y optimización.

V. MODELO DE DOMINIO Y CICLO DEL REPORTE

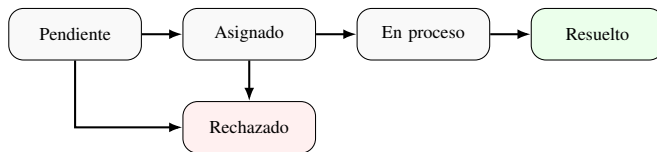
El elemento central del sistema es el reporte urbano. Su información no se limita a una descripción: también debe mantener la categoría, la ubicación, la evidencia, el ciudadano que lo creó, el supervisor asignado y el estado actual. Esta estructura permite consultar la evolución de una incidencia sin depender de mensajes externos o registros informales.

Cuadro IV: Datos principales de un reporte

Dato	Finalidad
Descripción	Explica brevemente el problema observado por el ciudadano.
Categoría	Clasifica la incidencia y permite asociarla con una estrategia de atención.
Ubicación	Guarda las coordenadas necesarias para mostrar el punto y calcular distancias.
Evidencia	Conserva la URL de la imagen subida al proveedor multimedia.
Estado	Representa la etapa actual del proceso de atención.
Asignación	Relaciona el reporte con el supervisor responsable.
Fechas	Permiten conocer cuándo se creó y cuándo se actualizó la incidencia.

V-A. Ciclo de vida

El reporte atraviesa una secuencia controlada. Al crearse se registra como pendiente. Luego puede asignarse a un supervisor, pasar a atención y finalmente marcarse como resuelto. También se contempla el rechazo cuando el reporte no corresponde a una incidencia válida. Las transiciones se controlan en el dominio para impedir cambios incoherentes.

**Figura 2:** Ciclo de vida simplificado de un reporte urbano.

V-B. Interacción entre módulos

Cuando se crea un reporte, el módulo de autenticación identifica al ciudadano, el módulo multimedia procesa la imagen y el módulo de reportes valida los datos antes de persistirlos. Posteriormente, el módulo de notificaciones comunica cambios de estado. Si el supervisor solicita una ruta, el módulo de optimización recibe las incidencias seleccionadas y devuelve una secuencia o estructura de atención.

Esta división evita que una sola clase concentre todas las responsabilidades. También permite probar cada parte con dependencias simuladas y reemplazar servicios externos mediante implementaciones alternativas.

VI. SEGURIDAD Y ACCESO A LA API

El acceso al backend se controla mediante Spring Security y tokens JWT. Después de un inicio de sesión válido, el servidor entrega un token que el cliente envía en las solicitudes protegidas. El backend valida la firma, recupera el rol y decide si el usuario puede ejecutar la operación solicitada.

La autorización por roles es necesaria porque un ciudadano no debe administrar supervisores ni cambiar libremente el estado de reportes ajenos. Del mismo modo, las operaciones administrativas deben mantenerse separadas de las funciones normales de registro y consulta.

Cuadro V: Ejemplos de operaciones de la API

Método	Recurso	Uso
POST	Autenticación	Registrar una cuenta o iniciar sesión.
POST	Reportes	Crear una incidencia con sus datos principales.
GET	Reportes	Consultar reportes según usuario, estado o asignación.
PATCH	Estado	Modificar el avance de atención de una incidencia.
POST	Multimedia	Subir una imagen y obtener su referencia.
POST	Optimización	Calcular una ruta o estructura para varios reportes.

Las credenciales de base de datos, la clave JWT y los datos del proveedor multimedia se manejan mediante variables de entorno. En una instalación real también sería necesario aplicar políticas de expiración de tokens, rotación de secretos, límites de solicitudes y registro de operaciones administrativas.

VII. COMUNICACIÓN Y SERVICIOS EXTERNOS

VII-A. Server-Sent Events

Server-Sent Events se utiliza para enviar actualizaciones desde el servidor hacia el cliente. La especificación permite que una página reciba eventos del servidor mediante una conexión HTTP persistente y una interfaz basada en EventSource [4]. Cuando un supervisor modifica el estado de una incidencia, el servidor puede notificar el cambio al ciudadano sin que la aplicación realice consultas repetitivas.

Este mecanismo resulta adecuado cuando la comunicación principal ocurre en una sola dirección. El cliente mantiene abierta una conexión y recibe eventos cuando existe información nueva.

VII-B. WebSocket

WebSocket se consideró para escenarios donde existe comunicación bidireccional. El protocolo establece un canal de comunicación persistente entre cliente y servidor después de una negociación inicial sobre HTTP [5]. En el proyecto puede utilizarse durante la navegación del supervisor, ya que la aplicación envía su ubicación y recibe información actualizada del recorrido.

La diferencia principal con SSE es que WebSocket permite que cliente y servidor envíen mensajes en ambos sentidos. Por esta razón, se adapta mejor a la actualización continua de posición y a cambios rápidos en una ruta.

VII-C. Almacenamiento de imágenes

Las imágenes se almacenan mediante Cloudinary. El backend no guarda directamente el archivo dentro de la entidad del reporte, sino que administra la subida y conserva la URL resultante. Esta decisión reduce el tamaño de la base de datos y permite cambiar el proveedor en el futuro mediante un adaptador.

VIII. VISTAS PRINCIPALES DEL SISTEMA

Para esta versión del artículo se incluyeron únicamente capturas del sistema en funcionamiento. Todas las imágenes se mantienen dentro del ancho de una sola columna para respetar el formato IEEE.

VIII-A. Panel del administrador

El administrador puede revisar reportes, asignarlos a supervisores y gestionar el personal del sistema. La Fig. 3 muestra únicamente el lado derecho del panel de administración, correspondiente a la gestión de usuarios y supervisores.

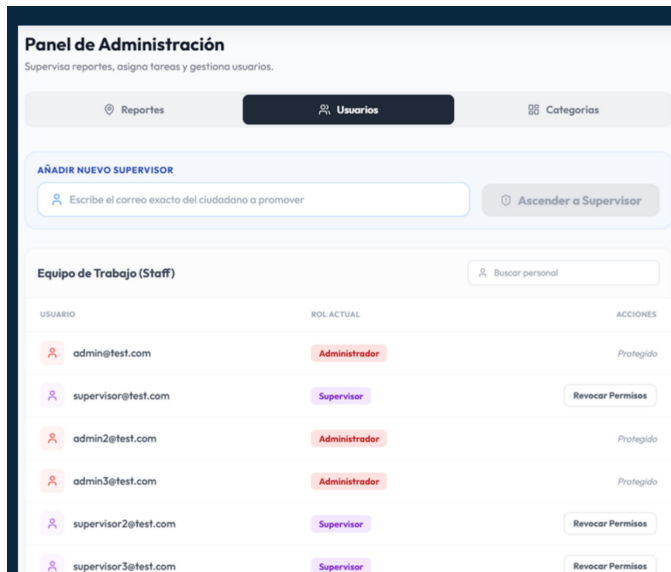


Figura 3: Gestión de usuarios y supervisores desde el panel de administración.

El administrador también puede gestionar las categorías disponibles. Cada categoría posee un nombre, una descripción y una asociación con el tipo de operación que utilizará el sistema. Esta configuración permite modificar el catálogo sin cambiar el código cada vez que aparece un nuevo tipo de incidencia.

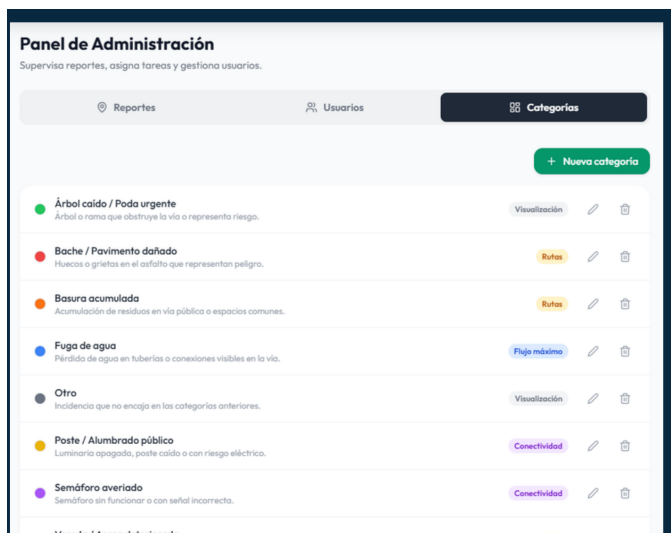


Figura 4: Mantenimiento de categorías urbanas.

VIII-B. Panel del supervisor

El supervisor dispone de una sección para consultar los reportes que tiene asignados. Desde esta vista puede revisar

la descripción, observar la evidencia y marcar el caso como resuelto cuando finaliza la atención.

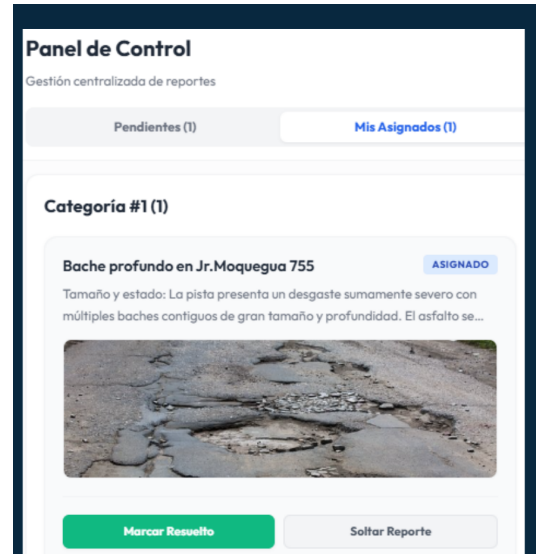


Figura 5: Detalle de un reporte asignado al supervisor.

La ruta completa se presenta en un mapa. Debido al límite de ancho de una columna, en este documento se priorizó la captura del detalle del reporte, que resulta más legible y representa mejor la interacción principal del supervisor.

IX. PATRONES DE DISEÑO APLICADOS

Durante el desarrollo se utilizaron patrones de diseño para resolver problemas concretos del proyecto. Los patrones clásicos descritos por Gamma et al. sirven como soluciones reutilizables para problemas frecuentes de diseño orientado a objetos [6]. No se buscó añadirlos de forma artificial, sino aplicarlos donde ayudaban a reducir dependencias o a organizar comportamientos que podían cambiar.

IX-A. Strategy

El patrón Strategy se emplea en el módulo de optimización. El problema principal consiste en que no todas las categorías requieren el mismo tipo de cálculo. Algunas incidencias necesitan una ruta ordenada, mientras que otras pueden asociarse a conectividad o flujo.

Para evitar un servicio con numerosos bloques condicionales, se definió una interfaz común de optimización y varias implementaciones. El servicio selecciona la estrategia correspondiente según la categoría y ejecuta el algoritmo mediante la misma operación. Esto permite añadir una nueva estrategia sin modificar las existentes.

En el proyecto se consideran implementaciones relacionadas con rutas, conectividad y flujo. La estrategia de rutas utiliza un enfoque de vecino más cercano ponderado; la estrategia de conectividad se apoya en Prim; y la estrategia de flujo utiliza Ford-Fulkerson con Edmonds-Karp. Estas implementaciones comparten una interfaz común y son invocadas por el servicio de optimización.

IX-B. State

El patrón State representa los estados posibles de un reporte. En lugar de concentrar todas las reglas en una sola clase con múltiples condiciones, cada estado controla las acciones que se permiten en ese momento.

Por ejemplo, un reporte pendiente puede ser asignado, mientras que un reporte resuelto ya no debería regresar libremente a cualquier estado anterior. Los estados considerados en el flujo incluyen pendiente, asignado, en proceso, resuelto y rechazado.

IX-C. Observer

Observer se utiliza para reaccionar ante eventos del sistema. Cuando se crea un reporte o se actualiza su estado, el componente que realiza la operación publica un evento. Otros componentes pueden escuchar ese evento y ejecutar acciones adicionales, como generar una notificación o registrar información de auditoría.

La ventaja es que el servicio principal no necesita conocer directamente a todos los receptores. Esto reduce el acoplamiento y permite agregar nuevos oyentes sin modificar el código que genera el evento.

IX-D. Adapter

Adapter permite integrar servicios externos mediante una interfaz propia del sistema. El dominio no trabaja directamente con las clases de Cloudinary o con la respuesta de un motor de rutas. En su lugar, utiliza puertos definidos por la aplicación y adaptadores que convierten los datos externos al formato esperado.

Este patrón es especialmente útil en una arquitectura hexagonal, porque mantiene los detalles técnicos fuera de la lógica principal. Si se reemplaza Cloudinary por almacenamiento local, el cambio se concentra en el adaptador.

IX-E. Builder

Builder se utiliza en la construcción de reportes. Una incidencia requiere varios datos: categoría, descripción, usuario, ubicación e imagen. El patrón permite reunir estos elementos paso a paso y validar que la información necesaria esté presente antes de crear el objeto final.

El uso de Builder hace que la creación sea más legible que un constructor con muchos parámetros. También evita generar objetos incompletos que luego causen errores durante la persistencia.

IX-F. Chain of Responsibility

Antes de guardar un reporte se ejecutan validaciones. En lugar de colocar todas las verificaciones dentro de un único método, se enlazan validadores que revisan aspectos distintos, como la ubicación o el contenido de la descripción.

Cada validador decide si el proceso continúa. Si una regla no se cumple, la cadena se interrumpe y se informa el error. Este enfoque facilita añadir una nueva validación sin modificar directamente las anteriores.

Cuadro VI: Resumen de patrones y propósito

Patrón	Propósito principal
Strategy	Cambiar el algoritmo de optimización sin alterar el servicio que lo utiliza.
State	Controlar las transiciones válidas del ciclo de vida de un reporte.
Observer	Informar eventos a varios receptores sin acoplarlos al emisor.
Adapter	Traducir servicios externos a interfaces propias del sistema.
Builder	Construir reportes completos y validados.
Chain	Ejecutar validaciones consecutivas antes de guardar una incidencia.

X. ALGORITMOS DE OPTIMIZACIÓN

El módulo de optimización agrupa algoritmos diferentes bajo la interfaz común utilizada por Strategy. La selección depende del tipo de categoría configurado por el administrador. Los algoritmos no pretenden resolver todos los problemas urbanos posibles, sino representar tres necesidades concretas del prototipo: ordenar visitas, conectar puntos y estimar flujo.

X-A. Vecino más cercano ponderado

Para categorías atendidas mediante una ruta secuencial, se parte de una ubicación inicial y se elige repetidamente el reporte no visitado con menor costo. El costo puede considerar la distancia y una ponderación asociada con la prioridad. El procedimiento termina cuando todos los puntos seleccionados han sido incorporados al recorrido.

Este enfoque es fácil de ejecutar y produce rápidamente una ruta utilizable, aunque no garantiza el óptimo global. Su propósito dentro del prototipo es entregar una secuencia práctica para el supervisor y permitir que posteriormente se reemplace por otro algoritmo sin modificar el resto del sistema.

X-B. Árbol de expansión mínima

Para incidencias asociadas con conectividad se utiliza el algoritmo de Prim. A partir de un nodo inicial, el algoritmo agrega en cada paso la arista de menor costo que conecta el conjunto ya visitado con un nuevo nodo. El resultado es una estructura que conecta todos los puntos sin ciclos y con costo total mínimo para el grafo considerado [7].

En el proyecto, esta estrategia sirve como representación de problemas donde interesa relacionar ubicaciones con la menor conexión total, por ejemplo incidencias de alumbrado o semáforos dentro de una zona.

X-C. Flujo máximo

Para la categoría de fugas de agua se incluyó una estrategia basada en Ford-Fulkerson con búsqueda Edmonds-Karp. El algoritmo identifica caminos aumentantes desde un origen hacia un destino y actualiza las capacidades residuales hasta que ya no existe un camino disponible [7].

La implementación funciona como una aproximación académica para demostrar que una categoría puede requerir una lógica distinta a una ruta de visitas. Para emplearla en una red hidráulica real se necesitarían datos técnicos de tuberías, capacidad y topología que no forman parte del alcance actual.

Cuadro VII: Relación entre categoría y estrategia

Necesidad	Algoritmo	Resultado
Ruta de atención	Vecino más cercano	Orden aproximado de visita de incidencias.
Conectividad	Prim	Árbol que conecta todos los puntos sin ciclos.
Flujo	Edmonds-Karp	Valor y caminos de flujo sobre la red modelada.
Visualización	Sin optimización	Presentación directa de puntos en el mapa.

XI. ESTRATEGIA DE PRUEBAS

Las pruebas se dividieron en varios niveles para cubrir tanto la lógica del backend como algunos flujos de uso del sistema. En el backend se validaron clases de servicio y reglas de negocio; en el cliente se verificaron acciones básicas de navegación y formularios; y finalmente se realizaron pruebas de carga sobre la API.

XI-A. Pruebas unitarias

JUnit se utilizó para comprobar reglas de dominio y servicios. Mockito permitió simular repositorios y dependencias externas, de manera que una prueba no requiera levantar toda la aplicación ni conectarse a servicios reales.

Este tipo de prueba resulta importante para validar comportamientos como transiciones de estado, selección de estrategias y validaciones previas al registro. La separación de la arquitectura facilita aislar estas clases.

XI-B. Pruebas funcionales y de carga

Cypress se utilizó para automatizar flujos desde la interfaz, como inicio de sesión, navegación y envío de formularios. Locust se empleó para enviar solicitudes concurrentes a la API y observar el comportamiento general de endpoints importantes cuando varios usuarios ejecutan operaciones al mismo tiempo.

Cuadro VIII: Herramientas de prueba

Herramienta	Uso en el proyecto
JUnit	Verificación de reglas y servicios principales del backend.
Mockito	Simulación de repositorios y dependencias durante las pruebas unitarias.
Cypress	Validación de inicio de sesión, formularios y navegación.
Locust	Simulación de usuarios concurrentes sobre endpoints de la API.

XI-C. Criterios de prueba

Las pruebas unitarias se concentran en comportamientos que pueden verificarse sin infraestructura: validación de reportes, cambios de estado, selección de estrategias y respuesta de los casos de uso ante datos inválidos. Para ello, los repositorios y adaptadores se sustituyen mediante objetos simulados.

Las pruebas funcionales verifican recorridos completos desde la interfaz. Un escenario básico incluye iniciar sesión, crear un reporte, revisar que aparezca en la lista y comprobar que un usuario sin permisos no acceda a una vista administrativa. Las pruebas de carga, por su parte, se enfocan en operaciones frecuentes como consulta y creación de reportes.

No se presentan cifras de rendimiento como resultados definitivos, debido a que dependen del equipo, la red y el entorno de despliegue. La función de Locust dentro de esta entrega es dejar preparado un mecanismo reproducible para ejecutar cargas y observar tiempos de respuesta, errores y concurrencia.

XII. DESPLIEGUE

La aplicación se preparó para un despliegue separado. El cliente web puede publicarse en Vercel, mientras que el backend se configura para ejecutarse en Render. La base de datos PostgreSQL se aloja en Supabase y las imágenes se mantienen en Cloudinary.

Esta separación permite actualizar cada parte sin desplegar nuevamente todo el sistema. También facilita administrar variables de entorno para credenciales, conexión a base de datos, claves JWT y configuración del proveedor de imágenes.

En un entorno local, el backend puede iniciarse con Maven y el cliente con Vite. Para un despliegue real sería necesario reforzar las políticas de seguridad, definir copias de respaldo y configurar monitoreo.

XIII. ESCENARIOS DE USO

Los escenarios de uso permiten observar cómo se conectan los módulos descritos anteriormente. No representan funciones adicionales, sino recorridos construidos a partir de las operaciones ya implementadas para cada rol.

XIII-A. Registro de una incidencia

El ciudadano inicia sesión y selecciona la opción para crear un reporte. Después elige una categoría, escribe una descripción, adjunta una fotografía y marca la ubicación sobre el mapa. Antes de guardar, el backend verifica que los datos obligatorios estén presentes y ejecuta la cadena de validaciones. Si la información es válida, la imagen se envía al proveedor multimedia y el reporte se registra con estado pendiente.

Una vez creado, el reporte puede consultarse desde la lista personal del ciudadano y desde las vistas autorizadas del personal. La ubicación se almacena como un punto geográfico, por lo que el mismo dato sirve para mostrar el marcador y para posteriores cálculos de distancia.

XIII-B. Asignación y atención

El administrador revisa los reportes pendientes y selecciona al supervisor responsable. Esta operación modifica la asignación y produce el cambio de estado correspondiente. El supervisor encuentra el reporte en su panel, revisa la descripción y la evidencia, y decide iniciar la atención.

Cuando el trabajo termina, el supervisor marca la incidencia como resuelta. La transición es procesada por el estado actual del objeto y se publica un evento para los componentes interesados. De esta manera, el módulo de notificaciones puede informar el cambio sin que el servicio de reportes dependa directamente de un mecanismo específico de comunicación.

XIII-C. Generación de una ruta

Cuando existen varios reportes de una misma categoría, el supervisor puede solicitar una optimización. El caso de uso recibe la ubicación actual y el conjunto de incidencias seleccionadas. Después consulta la categoría para determinar qué estrategia corresponde y delega el cálculo a la implementación registrada.

Para las categorías de recorrido, el resultado es una secuencia aproximada de visita. El backend devuelve los puntos ordenados y el cliente los representa sobre el mapa. El supervisor conserva la decisión final sobre el recorrido, ya que factores como cierres de calles, tráfico o restricciones operativas pueden no estar incluidos en los datos del prototipo.

Cuadro IX: Trazabilidad simplificada de escenarios

Escenario	Módulos	Resultado
Crear reporte	Auth, media, report	Incidencia pendiente con imagen y ubicación.
Asignar	User, report	Supervisor relacionado con el reporte.
Cambiar estado	Report, notification	Avance registrado y evento publicado.
Calcular ruta	Category, optimization	Secuencia o estructura para los puntos seleccionados.
Administrar categorías	Category	Catálogo editable y estrategia asociada.

XIII-D. Manejo de errores

Los errores se devuelven al cliente con un mensaje comprensible y un código HTTP adecuado. Por ejemplo, una solicitud sin autenticación debe rechazarse antes de ingresar al caso de uso, mientras que una transición inválida se detecta dentro del dominio. Las fallas de un proveedor externo, como la subida de una imagen, deben informarse sin registrar un reporte incompleto.

Esta separación ayuda a distinguir errores de entrada, autorización, reglas del negocio e infraestructura. También facilita escribir pruebas específicas para cada nivel y evita que una excepción técnica se muestre directamente al usuario final.

XIV. RESULTADOS Y DISCUSIÓN

Al finalizar el proyecto se logró implementar un flujo funcional que incluye registro e inicio de sesión, creación de reportes con ubicación e imagen, visualización geográfica, administración de categorías y usuarios, asignación de incidencias, cambio de estado y consulta de rutas.

La arquitectura utilizada ayudó a repartir el trabajo entre los integrantes. El cliente web y el backend pudieron avanzar de forma paralela mediante contratos de la API. Del mismo modo, la separación entre dominio e infraestructura permitió que varias clases pudieran probarse sin depender directamente de la base de datos.

Los patrones de diseño facilitaron la explicación técnica del sistema. Strategy representa la selección de algoritmos; State concentra las reglas del ciclo de vida; Observer desacopla las notificaciones; y Adapter mantiene los servicios externos fuera del dominio.

Entre las limitaciones se encuentra la necesidad de realizar pruebas con una mayor cantidad de reportes y usuarios. También sería conveniente medir tiempos de atención, evaluar el comportamiento del cálculo de rutas con conjuntos más grandes y mejorar el seguimiento GPS cuando el supervisor se desplaza.

XV. LIMITACIONES Y TRABAJO FUTURO

El prototipo demuestra el flujo principal, pero todavía existen aspectos que deberían evaluarse antes de utilizarlo en una entidad real. La primera limitación es la escala: se requiere probar el sistema con un volumen mayor de reportes simultáneos y con usuarios distribuidos en diferentes zonas de la ciudad.

La segunda limitación corresponde a la calidad de la información. Una fotografía y una ubicación no garantizan por sí solas que el incidente sea válido. Como trabajo futuro podría incorporarse moderación, detección de reportes duplicados y mecanismos para que otros ciudadanos respalden una incidencia existente.

También sería útil registrar métricas de operación como tiempo promedio de asignación, duración de atención, cantidad de reportes resueltos por categoría y reincidencias por zona. Estas métricas permitirían evaluar el sistema con información real en lugar de limitarse a demostrar que las funciones están implementadas.

Finalmente, el seguimiento de ubicación del supervisor debería considerar privacidad, consumo de batería y periodos de actualización. La aplicación no necesita guardar la posición continuamente fuera de una jornada activa. Estas decisiones deberán definirse junto con las reglas de uso de la organización que implemente la plataforma.

XVI. CONCLUSIONES

ReportaTuCalle ofrece una forma más ordenada de registrar y dar seguimiento a problemas urbanos. Frente a los medios informales, el sistema conserva en un solo lugar la ubicación, la evidencia, la categoría, el estado y el responsable de cada incidencia.

El proyecto permitió integrar conocimientos de desarrollo web, bases de datos, seguridad, geolocalización, patrones de diseño y pruebas. La arquitectura hexagonal resultó útil para separar la lógica de negocio de la infraestructura, mientras que los patrones aplicados resolvieron necesidades concretas del código.

En general, el prototipo constituye una base funcional para futuras ampliaciones. Con más pruebas, métricas y ajustes de seguridad, podría adaptarse a un contexto municipal o comunitario de mayor escala.

Repositorio del proyecto:

<https://github.com/Inka-Innovations-S-A-C/ReportaTuCalle>

REFERENCIAS

- [1] ISO/IEC/IEEE, "ISO/IEC/IEEE 29148:2018 Systems and software engineering—Life cycle processes—Requirements engineering," ISO, Geneva, Switzerland, 2018.

- [2] A. Cockburn, "Hexagonal architecture," Alistair Cockburn, 2005. [En línea]. Disponible en: <https://alistair.cockburn.us/hexagonal-architecture/>
- [3] PostGIS Project Steering Committee, "PostGIS documentation," 2026. [En línea]. Disponible en: <https://postgis.net/documentation/>
- [4] Mozilla Developer Network, "Using server-sent events," 2026. [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events
- [5] I. Fette and A. Melnikov, "The WebSocket protocol," RFC 6455, Internet Engineering Task Force, Dec. 2011.
- [6] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.
- [7] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.